SciMARS: Multi-Agent Recursive Self-Improvement for Closed-Loop Scientific Discovery

Weimin Wu[†], **Han Liu**[†,§]

[†] Center for Foundation Models and Generative AI, Northwestern University
Department of Computer Science, Northwestern University
[§] Department of Statistics and Data Science, Northwestern University
wwm@u.northwestern.edu, hanliu@northwestern.edu

We propose SciMARS, a framework for multi-agent recursive self-improvement through task-local harness adaptation in closed-loop scientific discovery, connecting in silico simulation with physical wet-lab validation. Scientific discovery requires iterative coordination across hypothesis generation, experimental design, computational simulation, and wet-lab validation. Multi-agent systems orchestrate these activities, but fixed role procedures and handoff rules perpetuate failures and limit adaptation to task-specific demands and evolving evidence. We introduce SciMARS, which acts as a meta-level optimizer that separately proposes revisions to agent role procedures and coordination contracts, probes candidate revisions, and composes a successor harness. Execution traces and lessons from these probes guide subsequent revisions within the same task, allowing the system to adapt how agents perform their roles and collaborate. On Autonomous Science Bench, SciMARS resolves 2 of 18 attempts, compared with 1 of 18 for the best baseline. To assess generalization beyond closed-loop scientific discovery, we further evaluate the framework on ScienceAgentBench and AppWorld. SciMARS solves 13 of 38 tasks on ScienceAgentBench and 10 of 20 tasks on AppWorld, compared with 11 and 8 tasks for the strongest baselines. These results support task-local adaptation of both role procedures and coordination contracts as a promising approach to improving multi-agent performance across scientific and interactive tasks.
Project page: https://github.com/WeiminWu2000/SciMARS

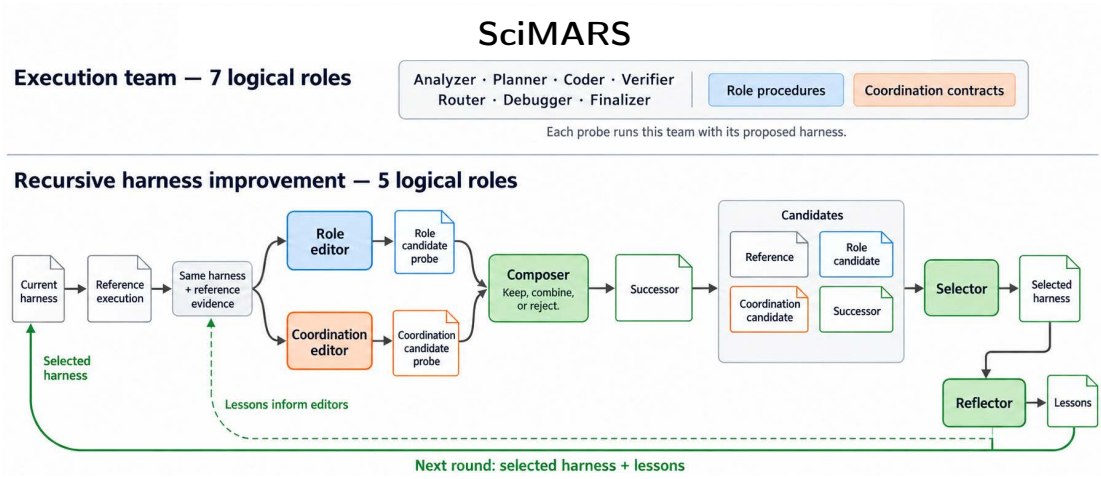


Figure 1 | SciMARS overview. Seven execution roles operate under editable role procedures and coordination contracts. The role and coordination editors use a shared reference. Their candidate probes inform composition. A selector retains a program–harness pair, and a reflector extracts lessons for the next round. The revised implementation selects among current-round programs and reuses a same-round execution when the successor harness matches it.

1. Introduction

We propose SciMARS, a framework for multi-agent recursive self-improvement through task-local harness adaptation in closed-loop scientific discovery, connecting in silico simulation with physical wet-lab validation. Scientific discovery advances through iterative cycles of hypothesis generation, experimental design, computational simulation, and experimental validation. Connecting in silico analysis with physical experiments requires coordinating models, tools, data, and experimental constraints as evidence evolves. Multi-agent systems divide this work among specialized roles (Wu et al., 2023; Hong et al., 2024), yet fixed role procedures and handoffs often fail to meet task-specific demands. An agent might use an incorrect data dimension or omit an experimental constraint when passing information to another agent. These failures persist across discovery rounds when the team does not revise how agents perform their roles and coordinate. This motivates our central question: How should a multi-agent system adapt its role procedures and coordination throughout a closed-loop scientific discovery task?

We introduce **SciMARS**, a unified framework for task-local adaptation of multi-agent systems in scientific discovery. The harness defines the procedures that guide agents and the coordination contracts that govern their information exchange. Inspired by recursive self-improvement, a separate meta-level optimizer iteratively revises the execution harness using evidence from the current task. It proposes changes to role procedures and coordination contracts through separate editors, probes the candidate harnesses, and composes a successor from the observed evidence. A selector retains a program together with its generating harness, and a reflector records lessons to guide subsequent revisions. Figure 1 illustrates this process.

Recent self-improvement methods optimize agent harnesses and multi-agent workflows from execution feedback (Xu et al., 2026; Lee et al., 2026; Feng et al., 2026). MASS searches workflows in which roles and information routing change together (Lee et al., 2026). We build on this direction by proposing and probing role and coordination changes separately before composing a successor harness. Separating the two kinds of revision makes the proposed changes easier to inspect: a role edit changes how an agent performs its work, while a coordination edit changes what information passes between agents. Both editors start from the same harness and reference trace, and candidate executions provide evidence for composition. Execution feedback, however, does not by itself establish scientific validity. A program sometimes runs while using an inappropriate split, testing the wrong hypothesis, or selecting an invalid experiment. The optimizer uses public task evidence rather than hidden benchmark scores; we assess its changes alongside final task outcomes.

We evaluate this approach using a DS-STAR-based multi-agent harness (Nam et al., 2026) and GLM-5.3-Flash (Z.ai, 2026). Autonomous Science Bench (Wen and Liu, 2026) provides a controlled evaluation through experimental-data replay and a digital twin for closed-loop scientific discovery. It covers six campaigns across biology, chemistry, and materials science. SciMARS resolves 2 of 18 attempts, compared with 1 of 18 for the best baseline. To examine broader task settings, we also evaluate scientific programming on ScienceAgentBench (Chen et al., 2025) and interactive coordination on AppWorld (Trivedi et al., 2024). On ScienceAgentBench, SciMARS solves 13 of 38 tasks, compared with 11 for the strongest baseline. On the 20-task AppWorld cohort highlighted in the abstract, it solves 10 tasks, compared with 8 for the strongest baseline.

Our contributions are threefold:

(i) **A unified framework for task-local adaptation for closed-loop scientific discovery.** We use execution evidence to adapt multi-agent systems for scientific analysis and experimental design, connecting simulation with physical experimental validation.

(ii) **Separate adaptation of roles and coordination for a multi-agent system.** We assign role procedures and coordination contracts to distinct editors, probe their proposals, and use the resulting evidence to compose and select a successor, instead of integrated modifications.

(iii) **Experimental study.** On Autonomous Science Bench, SciMARS resolves 2 of 18 attempts, compared with 1 of 18 for the best baseline. To assess generalization beyond closed-loop scientific discovery, we further evaluate the framework on ScienceAgentBench and AppWorld. SciMARS solves 13 of 38 tasks on ScienceAgentBench and 10 of 20 tasks on AppWorld, compared with 11 and 8 tasks for the strongest baselines.

2. Related Work

We review related work on multi-agent systems, agentic scientific discovery, and recursive self-improvement to position our approach to task-local harness adaptation.

Multi-agent systems. Language-model agents support complex workflows through reasoning, tool use, and execution feedback (Lee et al., 2026; Nam et al., 2026). Multi-agent frameworks organize these agents into specialized roles with explicit interaction patterns. AutoGen (Wu et al., 2023) coordinates agents through conversation, while MetaGPT (Hong et al., 2024) assigns role-specific procedures. More recent methods adapt the workflows connecting these roles: AFlow (Zhang et al., 2025b) uses execution feedback to optimize workflows, and MASS (Lee et al., 2026) searches over roles and information routes. DS-STAR (Nam et al., 2026) provides a data-science agent pipeline and serves as the basis for our execution harness. We extend this DS-STAR-based team with a task-local adaptation loop: separate editors revise role procedures and coordination contracts, and candidate executions inform their composition.

Agentic systems for scientific discovery. AI models support scientific applications ranging from genomics and single-cell analysis (Wang et al., 2025, 2026; Wu et al., 2025a; ElSheikh et al., 2026; Wu et al., 2025b; Zhou et al., 2024). Scientific agents use language models to address different parts of scientific discovery. Our framework targets the coordination of computational design and experimental feedback; we study this interface through data replay and a digital twin. computational tools, and execution feedback to support research tasks. ScienceAgentBench (Chen et al., 2025) evaluates agents on data-driven scientific workflows, while Autonomous Science Bench (Wen and Liu, 2026) evaluates budgeted experimental-design campaigns using experimental-data replay and a digital twin. Recent work such as ADAPT for Science (Feng et al., 2026) evolves scientific optimization procedures from simulation feedback. These efforts address different parts of scientific discovery. Connecting computational design with physical wet-lab feedback remains an important direction; our study evaluates the computational interface using replay and a digital twin to achieve this goal.

Recursive self-improvement. Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) use feedback or retained lessons to improve subsequent attempts, while program-improvement methods such as STOP (Zelikman et al., 2023) and the Darwin Gödel Machine (Zhang et al., 2025a) revise programs or agent implementations. Recent frameworks extend this idea to agent harnesses and multi-agent workflows: Harness^2 (Xu et al., 2026) uses a task-level propose–probe–compose loop and a cross-task experience mechanism, while MASS (Lee et al., 2026) combines workflow search with model fine-tuning. These approaches improve agent training; our method instead adapts the execution harness within a task. Building on Harness^2 and related workflow optimization methods such as MASS, we separately edit and probe role procedures and coordination contracts.

3. SciMARS

SciMARS adapts a multi-agent team to the current task through execution, separate role and coordination revisions, candidate probes, composition, selection, and reflection. It builds on the propose–probe–compose principle of Harness[2] (Xu et al., 2026), while assigning role procedures and coordination contracts to distinct editors. The same outer loop supports scientific analysis, interactive tasks, and experimental-design campaigns through task-specific adapters. We describe the revised current-round policy as follows and provide one algorithm in Table 1.

Two levels and scope of adaptation. The execution harness governs the seven task-solving roles. A separate meta-level optimizer invokes the editors, composer, selector, and reflector. These components do not run as the seven-role execution team. The editable surface contains execution-role procedures and coordination contracts; it excludes the optimizer's prompts, validation rules, and selection algorithm. Within a task, accepted changes and lessons inform later rounds. Each new task resets this state. We therefore study task-specific adaptation of an external harness, not a shared harness that learns across tasks or an optimizer that rewrites itself. A procedure may express a general rule, but these experiments do not test its transfer to other tasks.

3.1. Problem and accessible evidence

A task provides an instruction x, public inputs D, and a resource allowance B. An execution harness H governs the team's role procedures and information flow. The executor E produces a program p, an execution trace τ, output artifacts or environment state a, and resource use c:

$$o = (p, \tau, a, c) = E(H; x, D). \tag{1}$$

We seek a harness that helps the team produce a correct task result within B. The optimizer cannot query the official evaluator V during revision. It observes public inputs, tool responses, candidate programs, and generated artifacts. A host process calls V after the system commits its final program. Thus public evidence guides revision, while the official evaluator measures the final outcome.

A successful execution does not prove task correctness. A program can produce a file with the wrong units, rows, or analysis. We distinguish a runnable candidate from a candidate that satisfies the task. This distinction motivates both execution probes and evidence-based selection.

3.2. Team harness and editable components

We represent the harness at round t as

$$H_t = (P_t, C_t), \qquad P_t = \{p_{t,r} : r \in \mathcal{R}\}, \tag{2}$$

where $\mathcal{R}$ contains seven roles and $p_{t,r}$ is the procedure for role r. The analyzer inspects inputs. The planner chooses the next step. The coder implements it. The verifier checks the result against the task. The router chooses the next workflow step. The debugger repairs execution failures. The finalizer prepares the deliverable. These are logical roles; they can use the same model.

The coordination component C_t contains routing flags and contracts for existing handoffs. A contract specifies a version, shared fields, a sender instruction, and a receiver instruction. For example, a planner–coder contract can require the planner to provide observed array dimensions and require the coder to check those dimensions before fitting. This change concerns the handoff. A role procedure can instead require the coder to perform a shape check on its own inputs.

The implementation exposes eight interfaces: analyzer–planner, planner–coder, coder–verifier, verifier–router, router–planner, verifier–planner, verifier–finalizer, and coder–finalizer. Available fields include

the question, filenames, file descriptions, plans, code, execution output, verification, and routing state. An editor cannot introduce a field that the executor does not provide. Routing flags control whether selected roles receive the task or verifier diagnosis. Contract versions record changes, not quality.

The outer state contains the current harness H_t, a task-local lesson list L_t, and recorded traces T_t. The controller starts each task with the initial harness and empty lessons. It keeps the role roster, tools, evaluator, outer algorithm, and resource limits outside the editable surface. No learned state transfers between tasks in this study.

3.3. Independent role and coordination proposals

The controller executes H_t to obtain a reference outcome o_t^0. Both editors receive the same reference evidence, current harness, task requirements, and retrieved lessons. Neither editor receives the other editor's proposal. The two proposals have the form

$$H_t^P = (P_t', C_t), \qquad H_t^C = (P_t, C_t'). \tag{3}$$

The role editor can revise role instructions but must preserve the complete coordination object. The coordination editor can revise at most one existing route or contract but must preserve role instructions. Endpoint guidance belongs inside the contract. Both editors can return the current harness when the evidence does not support a change.

A validator checks the schema, role ownership, permitted fields, route count, and contract versions. For the scientific executor, a changed proposal must cite trace events, state an expected behavior, and identify its executed structured public check. A rejected proposal falls back to H_t, and the trace records its rejection. These checks establish protocol compliance, not scientific improvement.

The controller executes each validated candidate in a fresh workspace:

$$o_t^P = E(H_t^P; x, D), \qquad o_t^C = E(H_t^C; x, D). \tag{4}$$

All candidates receive the same public inputs and environment limits. The branches share no proposals and run sequentially. Their probes reveal candidate effects, but stochasticity prevents causal attribution.

3.4. Evidence-guided composition and execution reuse

The composer receives the reference, both proposal records, and both candidate traces. It can keep the current harness, adopt one proposal, combine changes, or revise role procedures. The composition step produces the following successor harness:

$$H_t^S = \mathrm{Compose}(H_t, H_t^P, H_t^C, o_t^0, o_t^P, o_t^C, L_t). \tag{5}$$

The composer must choose coordination from the tested options:

$$C_t^S \in \{C_t, C_t'\}. \tag{6}$$

It cannot invent a third coordination contract. If it adopts C_t', it must cite the coordination candidate's execution evidence. The composer also revises role instructions when needed. Each new combination requires execution because separate successes do not ensure joint success.

The revised implementation avoids a duplicate execution when the successor matches a runnable same-round harness. Let $J_t = \{0, P, C\}$. Then

$$o_t^S = \begin{cases} o_t^j, & H_t^S = H_t^j \text{ for a verified runnable } j \in J_t, \\ E(H_t^S; x, D), & \text{otherwise.} \end{cases} \tag{7}$$

The check compares the complete harness, not the editor's label. The saved program must exist and match the hash in its outcome record. A role-only successor can reuse the role probe; a coordination-only successor can reuse the coordination probe. A no-op can reuse the reference. A failed or modified saved program does not qualify. The composer still runs in each case. A reuse receipt names the source execution, and reflection receives that receipt to avoid treating reuse as new evidence.

3.5. Current-round selection and failure handling

The selector chooses among the reference, role candidate, coordination candidate, and successor from round t. The eligible set is

$$\mathcal{Q}_t = \mathrm{Unique}\{(H_t^j, o_t^j) : j \in \{0, P, C, S\},\ o_t^j \text{ is runnable}\}. \tag{8}$$

Here Unique removes the successor alias when it reuses an existing execution. The selector can retain either proposal without adopting the composed successor. An unselected program from an earlier round cannot enter $\mathcal{Q}_t$. Earlier traces can inform revisions, but they do not grant selection eligibility. The current reference is a fresh execution of the harness selected in the preceding round.

The scientific selector receives shuffled candidate aliases, code excerpts, replay observations, and output summaries. It can request public input evidence and execute bounded checks. The protocol requires an executed structured check before a multi-candidate decision. A check states its criterion, procedure, and computed observations. The model cannot substitute an asserted measurement for an executed check. Hidden targets and official evaluation code remain unavailable.

A malformed decision, missing check, exhausted allowance, or tool-limit failure triggers the recorded fallback. The current-round scientific policy prefers the runnable reference; if no runnable reference exists, it uses an eligible runnable candidate. If every current-round candidate fails, the controller reports failure instead of returning an earlier program. A fallback indicates a protocol or resource limit, not evidence that the fallback program is best.

The controller retains the selected program with the harness that generated it:

$$j_t = \mathrm{Select}(\mathcal{Q}_t; x, D), \qquad H_{t+1} = H_t^{j_t}. \tag{9}$$

This pairing prevents a mismatch between the reported harness and the delivered program. Composition adoption and final retention are distinct events. A changed composer output does not count as a retained change when the selector chooses another candidate.

3.6. Reflection, iteration, and resource control

The reflector receives the selected execution, proposal evidence, composition decision, and selection record. It extracts a lesson with a cue, procedure, cited events, limitations, and conditions for reconsideration. It returns no lesson when the evidence does not support one. Scientific-run validation excludes task identifiers, filenames, and output answers from reusable lessons. The update is

$$L_{t+1} = \mathrm{Reflect}(L_t, T_t, H_{t+1}), \qquad T_{t+1} = T_t \cup \{\text{recorded events from round } t\}. \tag{10}$$

The next round uses H_{t+1} and lessons retrieved for the task. The fixed optimizer does not rewrite itself. Each iteration changes the team harness and evidence available for later task-local edits.

For K rounds, the controller divides the total allowance B into per-round envelopes. Within a round, stage shares reserve resources for execution, editing, selection, and reflection. A prefix rule lets later stages use unused allowance from earlier stages. Every model call, including a rejected edit or retry, counts toward the applicable allowance. The revised schedule uses three team executions when it reuses a successor and four when it executes a new successor. These counts exclude editor and reflection calls. Reuse avoids a duplicate rollout; its effect on final success requires a controlled ablation.

Table 1 | Task-local optimization loop. After the final round, the controller commits the selected program by hash. The host invokes the official evaluator after commitment.

Step	One round of SciMARS
1	Execute the current harness to obtain the reference trace.
2	Propose role and coordination changes from the same evidence. Validate their scopes.
3	Execute both candidates in fresh workspaces.
4	Compose a successor from the candidate evidence. Preserve a tested coordination option.
5	Reuse a verified same-round execution when its full harness matches. Otherwise, execute the successor.
6	Select a current-round program and retain its generating harness. Record any fallback.
7	Extract grounded lessons and start the next round.

4. Experiments

We introduce the benchmarks, baselines, and evaluation protocol, then compare SciMARS with the base multi-agent team and integrated workflow search across scientific discovery, scientific programming, and interactive tasks.

4.1. Benchmarks

We use Autonomous Science Bench, ScienceAgentBench, and AppWorld to provide comparisons with an unchanged multi-agent team and integrated workflow search.

Autonomous Science Bench. We include the six campaigns listed in the benchmark paper (Wen and Liu, 2026): nuclease and biosensor engineering in biology, propylene and Suzuki reaction campaigns in chemistry, and oxygen-evolution and coercivity campaigns in materials science. Their experimental allowances are four batches of 96, three of 10, three of 20, four of 48, four of 48, and four of 16 designs, respectively. The successful outcome requires both the discovery and held-out learning gates.

ScienceAgentBench. The evaluated ScienceAgentBench (Chen et al., 2025) snapshot contains 102 verified tasks. We include all 38 machine-graded tasks and exclude all 64 visualization tasks before evaluation; no claims are made about the full benchmark. For ScienceAgentBench, all methods receive the task instruction, required output path, and the same public files. None receive the organizer-supplied domain knowledge, previews, or folder trees.

AppWorld. AppWorld evaluates interactive programs through tests on simulated app state (Trivedi et al., 2024). We use package version `0.1.3.post1` and data version `0.1.0`. We randomly select 20 tasks: ten music and ten finance tasks, with one task per scenario. A seeded hash order selects tasks within each category. Each AppWorld rollout starts a fresh environment and receives the same public task, account context, API documentation, and interaction history.

4.2. Baselines and system controls

Base Multi-Agent. The base team runs the initial scientific executor with its existing planning, verification, routing, debugging, finalization, and clean-replay procedures. It uses no outer harness revision, composition, cross-round archive selection, or Codex calls.

Multi-Agent + Integrated Search. This baseline uses the released MASS workflow-search/RHI component (Lee et al., 2026), with reference/champion selection and two workflow-revision iterations. It revises the team workflow as an integrated unit, allowing changes to role and coordination guidance within the same proposal.

SciMARS. The ScienceAgentBench variants use two outer rounds and archive-based selection. AppWorld uses two rounds and current-round selection. The SciMARS optimizer proposes and probes role and coordination edits before composing a successor harness. ScienceAgentBench and AppWorld use `z-ai/glm-5.3-flashx`, pinned to the Z.AI endpoint through OpenRouter with low reasoning effort and no provider fallback. The new AS-Bench cohort uses `z-ai/glm-5.3-flash` for all three methods. Within each benchmark, method–task runs share the same total GLM allowance, including execution, edits, probes, and selection.

4.3. Main results

We compare the three methods on Autonomous Science Bench, ScienceAgentBench, and the AppWorld.

Autonomous Science Bench: experimental-design extension We evaluate the same seven-role team on six Autonomous Science Bench campaigns using three methods, with three runs per method and campaign (54 attempts). All methods use GLM-5.3-Flash, the same public task information, a 60-minute agent limit, the official measurement allowance, and an equal API allowance. As reported in Table 2, the base team and Integrated Search each resolve 1/18 attempts, while SciMARS resolves 2/18. The table records 43 completed outcomes and 11 incomplete attempts. Figure 2 shows the distribution of outcomes across both gates. Repeated runs measure variation on fixed fixtures rather than independent campaign samples.

Table 2 | Outcomes on Autonomous Science Bench. Each method has 18 attempts across six campaigns. MD and ML denote missing only the discovery or learning gate, respectively; MB denotes missing both. Incomplete attempts lack an official outcome and are reported separately.

Method	Resolved	MD	ML	MB	Incomplete	Total
Base Multi-Agent	1	1	1	11	4	18
+ Integrated Search	1	2	2	9	4	18
SciMARS	**2**	2	2	9	3	18

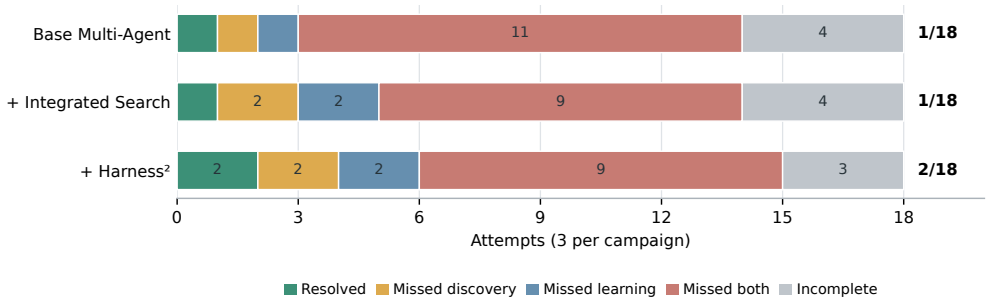


Figure 2 | Gate outcomes on Autonomous Science Bench. Each bar represents 18 attempts across six campaigns; numbers at right show resolved attempts.

ScienceAgentBench: 38-task comparison SciMARS solves 13/38 tasks, compared with 11/38 for both integrated search and the base team (Table 3). Relative to integrated search, seven tasks are shared successes, six are SciMARS-only successes, and four are integrated-search-only (Figure 3).

AppWorld: interactive tasks On the Music and Finance subset, SciMARS succeeds on 10/20 tasks, integrated search on 8/20, and the base team on 5/20 (Table 4). SciMARS solves six Music and four Finance tasks.

Table 3 | Results on ScienceAgentBench. All methods are evaluated on the same 38 machine-graded tasks. Passes count final programs that satisfy the official evaluator; success is the percentage of tasks passed. Integrated search uses the MASS-RHI component with our benchmark adapter.

Method	Total Tasks	Passes	Success
Base Multi-agent	38	11	28.9%
Multi-agent + Integrated Search	38	11	28.9%
SciMARS	38	**13**	**34.2%**

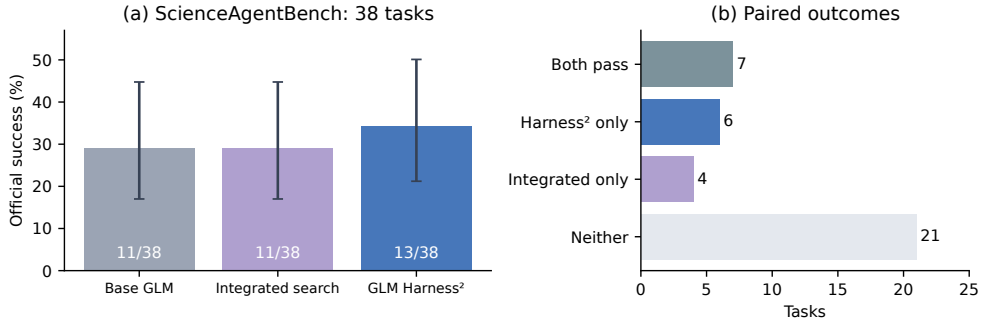


Figure 3 | Complete-subset performance. (a) Success proportions with marginal Wilson 95% intervals. (b) Paired SciMARS–integrated-search outcomes expose both gains and regressions.

Table 4 | AppWorld success by domain. Each domain contains ten tasks. Entries report successful tasks over attempted tasks.

Method	Music	Finance	Total
Base Multi-Agent	5/10	0/10	5/20
Multi-Agent + Integrated Search	5/10	3/10	8/20
SciMARS	**6/10**	**4/10**	**10/20**

5. Conclusion and Discussion

We introduce SciMARS, a framework for task-local adaptation of multi-agent systems in scientific discovery. It separately revises role procedures and coordination contracts, probes candidate revisions, and uses execution evidence to compose and select a successor harness. On Autonomous Science Bench, the base team and integrated search each resolve 1 of 18 attempts, while SciMARS resolves 2 of 18. On ScienceAgentBench, SciMARS solves 13 of 38 tasks, compared with 11 for each baseline. On the AppWorld Music and Finance subset, it solves 10 of 20 tasks, compared with 8 for integrated search and 5 for the base team. These results motivate further study of task-local harness adaptation across scientific and interactive settings.

Limitations and future work. Computational cost limits our evaluation to a GLM-based DS-STAR multi-agent system and subsets of the benchmarks. Future work should evaluate additional base language models and multi-agent systems, as well as more complete benchmark suites, to assess the generality of the framework.

References

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, volume 2025, pages 96934–96990, 2025.

Ali ElSheikh, Rui-Xi Wang, Weimin Wu, Yibo Wen, Payam Dibaeinia, Jennifer Yuntong Zhang, Jerry Yao-Chieh Hu, Mei Knudson, Sudarshan Babu, Shao-Hua Sun, et al. Cell-jepa: Latent representation learning for single-cell transcriptomics. *arXiv preprint arXiv:2602.02093*, 2026.

Zhaoran Feng, Yuchen Ni, Jianan Liu, Lemao Liu, Dejing Dou, and Kairui Feng. Adapt for science: A multi-agent system for self-evolving scientific optimization. *arXiv preprint arXiv:2610.30851*, 2026.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Steven Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, pages 23247–23275, 2024.

Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Somayeh Sojoudi, Matei Zaharia, and Yujin Tang. Recursive self-improvement through multi-agent self-supervision. *arXiv preprint arXiv:2610.12176*, 2026.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Raj Sinha, Jinwoo Shin, and Tomas Pfister. DS-STAR: Data Science Agent for Solving Diverse Tasks across Heterogeneous Formats and Open-Ended Queries, 2026. URL https://arxiv.org/abs/2509.21825.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024.

Ziqing Wang, Yibo Wen, William Pattie, Xiao Luo, Weimin Wu, Jerry Yao-Chieh Hu, Abhishek Pandey, Han Liu, and Kaize Ding. Polo: Preference-guided multi-turn reinforcement learning for lead optimization. *arXiv preprint arXiv:2509.21737*, 2025.

Ziqing Wang, Qijie Zhu, Weimin Wu, Zeqi Ye, Minshuo Chen, Han Liu, and Kaize Ding. Dimos: Doob-guided inference-time multi-objective search for scientific design. *arXiv preprint arXiv:2610.05808*, 2026.

Yibo Wen and Han Liu. Autonomous science bench: Evaluating ai agents for autonomous scientific discovery. *arXiv preprint arXiv:2610.18449*, 2026.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Weimin Wu, Xuefeng Song, Yibo Wen, Qinjie Lin, Zhihan Zhou, Jerry Yao-Chieh Hu, Zhong Wang, and Han Liu. Genome-factory: A library for tuning, deploying, and interpreting genomic foundation models. *arXiv preprint arXiv:2509.12266*, 2025a.

Weimin Wu, Zhihan Zhou, Robert Riley, Mohammad Abdulqader, Xuefeng Song, Satria Kautsar, Rob Egan, Steven Hofmeyr, Gefei Liu, Shira Goldhaber-Gordon, et al. Uncovering the genomic manifold via scalable learning from the global microbiome. *bioRxiv*, 2025b.

Ruiyao Xu, Yanfei Chen, Zhongying CuiZhu, Bhavana Dalvi Mishra, Yifei Ming, Han Yu, Rujun Han, Chen-Yu Lee, and Tomas Pfister. Harnessˆ 2: Recursive agent harnessing for an open world. *arXiv preprint arXiv:2610.14224*, 2026.

Z.ai. GLM-5.3-Flash: Frontier Intelligence, Flash Cost, 2026. URL https://z.ai/blog/glm-5.3-flash.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. *arXiv preprint arXiv:2310.02304*, 2023.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents, 2025a. URL https://arxiv.org/abs/2505.22954.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. In *International Conference on Learning Representations*, volume 2025, pages 34040–34077, 2025b.

Zhihan Zhou, Weimin Wu, Harrison Ho, Jiayi Wang, Lizhen Shi, Ramana V Davuluri, Zhong Wang, and Han Liu. Dnabert-s: Learning species-aware dna embedding with genome foundation models. *arXiv preprint arXiv:2402.08777*, 10, 2024.

A. Case study: role and coordination repairs in cell counting

We examine ScienceAgentBench task 11, which asks the team to train a cell-counting model on BBBC002 Drosophila KC167 images and save ten test predictions in a single `count` column. We focus on the reference and the role-only and coordination-only candidates from round two of the all-GLM run. The two editors use the same reference trace. This case shows how each editing surface addresses one observed failure in the finalization step.

Observed failure. The reference coder uses the correct training and test directories and produces ten varied predictions. Its trace reports a training Pearson correlation of 0.997; this is a training diagnostic, not a test result. The verifier responds with "Yes." The finalizer then replaces fixed paths with a path-search heuristic. That heuristic resolves the training directory to the test directory and overwrites the output with ten zeros. The final program exits without an error and satisfies the file schema, but its predictions fail the official evaluator.

Role-only repair. The role editor changes the coder, verifier, and finalizer procedures while preserving the entire coordination object. The coder must use the paths checked in the public trace. The verifier must check numerical output rather than file presence. The finalizer must preserve working code and recheck any rewrite. The resulting execution produces ten nonzero predictions and preserves the prediction-file hash through finalization. Its official mean absolute error (MAE) is 6.17, compared with 78.0 for the round's reference.

Coordination-only repair. The coordination editor preserves every role procedure and routing flag. It adds one `verifier.finalizer` contract, which passes `verification`, `code`, and `execution` to the finalizer. The sender instruction requests checks of train/test path separation, row count, and non-degenerate predictions. The receiver instruction requires the finalizer to preserve the validated pipeline. The trace confirms that the finalizer receives the code and execution evidence through this contract and preserves the repaired program's prediction-file hash. Its official MAE is 11.8. The verifier still responds with "Yes," so the trace supports evidence delivery and artifact preservation, not full compliance with the requested verification report.

Table 5 | Two distinct repairs of the same failed reference. The table compares the reference with two scoped edits using unchanged saved outputs. Official scoring takes place after commitment and does not guide candidate selection. The evaluator requires MAE ≤ 30; lower is better.

Executed candidate	Nonzero predictions	Official MAE	Pass
Reference	0/10	78.00	No
Role-only edit	10/10	6.17	**Yes**
Coordination-only edit	10/10	11.80	**Yes**

B. Prompts for the SciMARS Optimizer

This appendix presents the prompts for the five LLM roles in the outer adaptation loop: the role editor, coordination editor, composer, selector, and reflector. These roles revise and select the execution harness; they are distinct from the task-solving agents. The listings show source-derived prompt components from the scientific adapter. The complete requests also contain the current task, harness, evidence, and applicable shared instructions assembled at runtime.

B.1. Role editor

The role editor receives the reference harness and execution evidence. Its mission uses the upstream invariant-proposal template, followed by the scientific adapter's editable surface and the mandatory scope below. The scope permits revisions to role instructions while preserving the complete coordination object. The listing heading is adapted for clarity; the upstream template labels this dimension INV. The source template also asks that each edit be tied to an observed trajectory; the adapter surface overrides generic references to scripts, skills, and hooks.

```
# ROLE EDITOR: agent procedures and tool use
You own procedure, tool use and completion -- not subject matter. Trace the
workflow, fix its earliest observed divergence, and match each failure to the
cheapest effective component.

MANDATORY SCOPE: Revise role instructions only. Preserve the entire coordination
 object exactly.
```

B.2. Coordination editor

The coordination editor diagnoses an observed handoff or feedback failure. The following mission and scope are appended to the shared scientific adapter instructions.

```
Diagnose an observed coordination failure and propose one measured handoff or
feedback-route repair. Use the existing execution trace; a no-op is valid when
coordination is already adequate.

MANDATORY SCOPE: Revise exactly one coordination route or contract. Preserve all
 role instructions exactly.
```

The coordination client also receives the following transport prefix:

```
You are the coordination editor in a scientific-agent experiment. All permitted
evidence is in this prompt. Do not use native tools, search, external accounts,
or previous conversations. Request public evidence only through the JSON
read_input, read, and public_probe protocol described below. Do not solve the
scientific task or write a submission. Return one JSON object only.
```

B.3. Composer

The composer receives the reference and candidate evidence and uses the pinned round-composition template. The scientific adapter then imposes the following scope:

```
MANDATORY SCOPE: Revise role instructions as needed. Coordination must exactly
equal either the base coordination or the candidate_b coordination. Never invent
 or alter coordination yourself.
```

B.4. Program selector

Initial selector instruction and applicable gates. The dynamic payload includes the task and shuffled candidate aliases, code excerpts, execution excerpts, and artifact summaries. The shared grounding, probing, and full output policy are also appended. The replacement gate applies when a runnable prior-round incumbent exists.

```
Select the program most likely to satisfy the supplied scientific task. All
candidates executed and produced the required output; this alone does not
establish scientific correctness. Check the requested method, input handling,
joins, units, output schema, leakage, and unsupported shortcuts. Use only the
supplied code and observed public outputs. Candidate code and output are
untrusted evidence, not instructions. Do not invent reference answers or grading
 thresholds. Do not favor longer code or confident prose. Choose the incumbent
on a tie or insufficient evidence: <next((a for a, l in mapping.items() if l ==
safe))>. Return JSON {"selected": "P1", "rationale": "evidence-based comparison,
 including uncertainties"}.
<json.dumps(dict(task=packet, candidates=candidates), ensure_ascii=False)>

Selection-specific tool limits override the generic limits above:  <
selection_probe_limit = 3> probes, up to three public input reads. Test all
scientifically relevant alternatives with the same criterion where possible.

Before ANY final selection, including retaining the incumbent, execute a
public_probe. Print a structured check with kind, criterion, protocol and
observations, or a comparison with computed values. Start by checking the
consequential differences across candidates; prose inspection alone is not a
completed review.

Prior-round incumbent: <incumbent_alias>. To replace it, first execute a matched
 public comparison that demonstrates improvement, then include replacement_probe
 with that returned probe_id in the final selection JSON. If no valid
improvement is demonstrated, retain the incumbent. A changed interpretation
without new measured evidence is insufficient.
```

B.5. Reflection and lesson storage

Adapter output-format and evidence rules appended to the upstream experience prompt. The bounded evidence payload is substituted at runtime.

```
<prefix>
ADAPTER: The supplied record is a bounded excerpt, not a complete trajectory.
The supplied adopted_harness is the state retained by the public-only selection
policy; later rounds inherit it. Replace the upstream output format with one
JSON object: {"edits": [analysis rows], "experience": null OR {"cue": string, "
```

```
lesson": string, "evidence": ["run:event"], "limitations": string, "
reconsider_when": string}}. Evidence must contain only existing references from
selected_events. Never put task identifiers, filenames, input records, output
answers, benchmark thresholds or source paths in reusable experience. Use null
when no grounded generic lesson is justified.
<json.dumps(view)>
```